

# Bloque 14 - Nivel avanzado: decidir con evidencia imperfecta

## Propósito

Este bloque reúne problemas que no se resuelven con una gráfica ni con una consulta: una caída de conversión que podría ser producto, marketing o un error de medición; un intervalo incierto; una alerta que debe acabar en una acción; y datos que ya no caben cómodamente en un archivo local.

El caso continuo es **Lumen**, una app de reservas. El 8 de junio la conversión de visita a reserva baja de 4,8 % a 3,6 %. El equipo quiere saber si el nuevo formulario la causó, cuánto confiar en la estimación, cuándo alertar y cómo analizar eventos masivos y datos externos sin crear nuevas fugas o riesgos de privacidad.

## Resultados observables

Al terminar podrás formular un estimando causal y su contrafactual, dibujar un DAG y nombrar sus supuestos; construir un intervalo bootstrap y un análisis de sensibilidad; convertir una anomalía en una alerta con runbook; y consultar datos particionados con criterio. También podrás extraer una API paginada de forma reproducible y tratar coordenadas sin confundir sistemas de referencia ni exponer información personal.

## Prerrequisitos

Conviene haber cursado estadística, SQL, métricas y series temporales. No se presupone experiencia con Parquet, DuckDB, APIs ni sistemas de referencia de coordenadas: se presentan desde el problema que resuelven.

## Lecciones

1. Causalidad: contrafactuales, DAG y diseños
2. Bootstrap, incertidumbre y sensibilidad
3. Anomalías, alertas y runbooks
4. Escala: Parquet, particiones y DuckDB
5. APIs, datos geoespaciales y fuentes externas

## Práctica integrada

Realiza el [laboratorio de investigación de la caída](#) antes de consultar la [solución razonada](#). El script [14-caida-conversion.py](#) ilustra los cálculos de bootstrap y el diseño de una alerta, pero no sustituye el razonamiento causal.

# Causalidad: contrafactuales, DAG y diseños

## Resultado y prerequisites

Al terminar podrás convertir “el formulario nuevo bajó la conversión” en una pregunta que se pueda investigar, declarar el efecto que buscas y elegir un diseño proporcional a la decisión. Necesitas distinguir una tasa de conversión de una causa; no necesitas haber usado un modelo causal.

## El problema: dos explicaciones para el mismo descenso

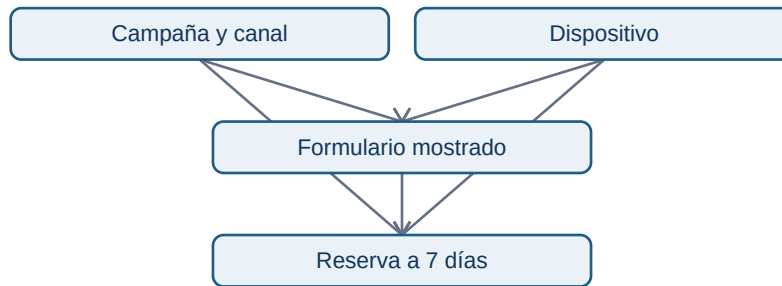
En Lumen, el formulario B se activó el 8 de junio. A partir de entonces la conversión observada bajó. Eso describe una **asociación temporal**: dos hechos ocurrieron juntos. La pregunta causal es distinta: \*¿cuánto habría cambiado la conversión de esas mismas visitas si B no se hubiera mostrado?\* Ese resultado alternativo, no observable para la misma visita en el mismo instante, se llama **contrafactual**.

Definimos el estimando antes de mirar el resultado: diferencia media de conversión a 7 días entre mostrar B y mostrar A a las visitas elegibles entre el 8 y el 21 de junio. La población, la ventana, la unidad (visita, no evento) y el horizonte cambian la pregunta. “Subió el uso” no es un estimando.

Una campaña de pago empezó el mismo día y trae visitas menos propensas a reservar. También cambió el navegador móvil de parte de la audiencia. Ambas variables pueden explicar simultáneamente qué formulario vio una persona y si reservó: son **confusores**.

## Un DAG hace explícita la historia que estás suponiendo

La pregunta es: ¿por qué una comparación bruta puede engañar? El siguiente grafo dirigido acíclico (DAG) no prueba causalidad; obliga a declarar qué caminos se deben bloquear.



El camino `Formulario <- Campaña -> Reserva` no representa el efecto del formulario: mezcla audiencia con experiencia. Medir canal y dispositivo puede permitir ajustar; no medir una causa relevante impide prometer que el ajuste “ha eliminado el sesgo”. No controles una variable que ocurre **después** del tratamiento, como “tiempo dentro del nuevo formulario”: podría ser un mediador y cambiar la pregunta.

## Diseños: qué comparan realmente

Un experimento A/B asigna A o B al azar antes de la experiencia. La aleatorización hace comparables, en expectativa, variables conocidas y desconocidas. Requiere asignación estable, análisis por intención de tratar, instrumentación válida y guardrails (errores, cancelaciones, latencia). No autoriza parar al primer resultado llamativo ni ignorar interferencias entre usuarios.

Si no se puede experimentar, el diseño cuasiexperimental intenta construir un contrafactual aproximado, nunca automático:

Para Lumen, el diseño preferible es A/B por visita elegible, estratificado por plataforma si el equipo lo necesita operativamente. Si B se desplegó a toda la población, una comparación antes/después **no separa** campaña, estacionalidad y formulario; una diferencia en diferencias con un país no desplegado solo es defendible tras mostrar tendencias previas semejantes y cambios comparables.

## Ejemplo trabajado: lectura prudente

Supón 20.000 visitas A y 20.000 B. A convierte 4,8 % y B 4,2 %: diferencia B-A = -0,6 puntos porcentuales. Esa es la estimación observada, no “B destruye 12,5 % de las reservas”. La siguiente lección calcula su incertidumbre. Si la asignación fue defectuosa o B solo se mostró en móvil, el número responde a otra pregunta.

## Errores frecuentes

- Ajustar por todas las columnas disponibles sin dibujar el momento causal de cada una.

- Llamar “control” a usuarios que nunca pudieron recibir B; viola comparabilidad.
- Confundir significación estadística con impacto de producto: -0,05 puntos puede ser muy preciso y aun irrelevante.
- Ocultar cambios de definición del evento `reserva_confirmada` entre variantes.

## Resumen y comprobación

Una afirmación causal necesita tratamiento, resultado, población, contrafactual y supuestos. Un DAG es un mapa de supuestos, no una máquina de verdad.

1. ¿Qué contrafactual falta en una comparación antes/después?
2. ¿Por qué campaña puede ser confusor y tiempo de formulario un mal control?
3. ¿Qué prueba previa pedirías antes de aceptar diferencias en diferencias?

Aplica la formulación al [laboratorio integrado](#).

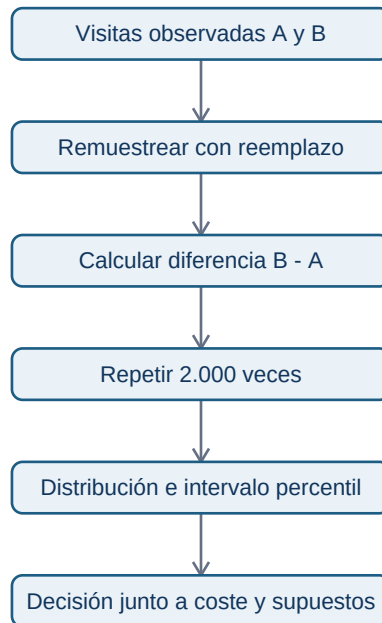
# Bootstrap, incertidumbre y sensibilidad

## Resultado y prerequisites

Podrás construir e interpretar una distribución bootstrap para una diferencia de conversión y separar incertidumbre de muestreo de sesgo causal. Debes saber calcular una media o proporción.

## De un número a una distribución de números

Lumen observa una diferencia B-A de -0,6 puntos porcentuales. Una muestra alternativa de visitas habría dado una cifra algo distinta. El **bootstrap** aproxima esa variación: toma muchas muestras del mismo tamaño, con reemplazo, de los datos observados; recalcula la estadística en cada una; y usa la distribución resultante para describir estabilidad.



“Con reemplazo” significa que una visita puede aparecer dos veces en una réplica y otra ninguna. No inventa usuarios nuevos ni corrige el sesgo de selección. Si los eventos de una persona están repetidos, la unidad de remuestreo debe ser la persona o el clúster, no cada evento; de lo contrario se finge más información de la que existe.

## Ejemplo mínimo y lectura

Para cada réplica, muestrea 20.000 conversiones de A y 20.000 de B, calcula `proporcion_B - proporcion_A`, y guarda el resultado. Si los percentiles 2,5 y 97,5 son -0,95 y -0,23 puntos, un intervalo bootstrap percentil al 95 % compatible con este procedimiento es  $[-0,95, -0,23]$ . No significa “hay 95 % de probabilidad de que el efecto verdadero esté dentro” sin especificar un marco estadístico; sí comunica que con este modelo de remuestreo el efecto negativo no es frágil al azar muestral.

La decisión requiere magnitud: si perder 0,23 puntos ya supera el guardrail, se pausa B. Si el intervalo incluye un daño pequeño e impacto esperado muy bajo, se puede ampliar muestra. Reporta denominadores, fecha de corte, variantes excluidas y si el intervalo fue planeado antes de mirar.

## Sensibilidad: hacer visibles decisiones que cambian el veredicto

La sensibilidad pregunta “¿seguiría la recomendación bajo alternativas defendibles?”. Para Lumen construye una tabla: ventana de 7 frente a 14 días; incluir/excluir tráfico de afiliados etiquetado tarde; métrica por visita frente a usuario; y ajuste por plataforma. No elijas alternativas después para fabricar

una conclusión.

Si la conclusión cambia de “daño claro” a “sin efecto” por una limpieza defendible, la conclusión correcta es fragilidad y necesidad de auditar, no escoger la fila favorita. El bootstrap tampoco arregla un evento duplicado, atribución errónea o confusor no medido.

## Mini-laboratorio

Ejecuta `python notebooks/practicas/14-caida-conversion.py`. Compara el intervalo de la diferencia y modifica la semilla o la tasa de B. Después explica qué pregunta **no** responde el código: no demuestra que B cause el efecto porque los datos simulados no representan el mecanismo de asignación real.

## Resumen y comprobación

Bootstrap cuantifica variación al remuestrear los datos disponibles. Sensibilidad expone la dependencia de decisiones y supuestos; ninguna reemplaza un diseño causal.

1. ¿Qué unidad remuestrearías si cada usuario genera muchas visitas?
2. ¿Por qué un intervalo estrecho puede coexistir con un resultado sesgado?
3. Nombra una alternativa de definición de métrica que probarías.

# Anomalías, alertas y runbooks

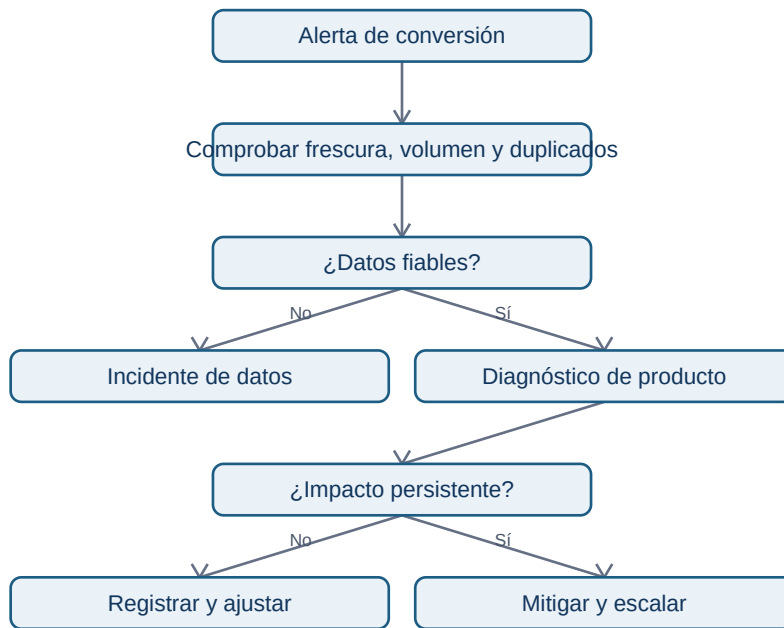
## Resultado y prerequisites

Podrás diferenciar una observación rara de un incidente, definir una alerta accionable y escribir el primer tramo de su runbook. Se asume que conoces una métrica y su denominador.

## Una alerta no es una línea roja

Una **anomalía** es un valor que se aparta de un patrón esperado. Un descenso de conversión puede ser producto, estacionalidad semanal, campaña, falta de datos o una definición cambiada. Una **alerta** es una regla que pide a una persona actuar porque el coste de no detectar algo supera el coste de investigarlo. El detector no diagnostica por sí solo.

Para Lumen se define: “alertar a la persona de guardia si la conversión diaria por plataforma está 25 % por debajo de la referencia comparable durante dos ventanas consecutivas, con al menos 1.000 visitas y frescura menor de 90 minutos”. La referencia debe ser explícita: mediana de los cuatro mismos días de semana anteriores, no una media de todo el mes que mezcle fin de semana y laborable.



El flujo evita un error común: comunicar “la conversión cayó” a dirección cuando en realidad el SDK dejó de enviar eventos Android. El primer paso es validar la observabilidad.

## Runbook mínimo, responsable y evidencia

Un **runbook** es una instrucción operativa para responder de forma repetible. Debe existir antes de alertar. Para esta señal incluye:

1. Propietario y horario de cobertura; canal de escalado y severidad.
2. Enlace a consulta versionada: numerador, denominador, zona horaria y retraso esperado.
3. Comprobaciones de calidad: frescura, conteo de eventos fuente, nulos, duplicados y cambios de esquema.
4. Cortes de diagnóstico: plataforma, versión de app, canal, país y experimento; evitar segmentar hasta encontrar ruido.
5. Contexto operativo: despliegues, campañas, precios, stock y cambios de tracking.
6. Acción reversible y criterio de cierre: pausar flag, corregir instrumentación, o documentar efecto esperado.

Guarda cada alerta con hora, valor, referencia, versión de regla, persona que cerró y causa final. Esa etiqueta permite estimar precisión operativa: cuántas alertas eran incidentes reales frente a ruido. Un umbral más sensible sube detección pero también fatiga; una alerta ignorada repetidamente es una deuda de confianza.

## Límite de un z-score y alternativa práctica

Un umbral “menos de 3 desviaciones estándar” presupone una distribución y estabilidad que la conversión diaria rara vez tiene: cambia con día de semana, campañas y tamaño de muestra. Para empezar, una referencia estacional explícita, mínimo de volumen y persistencia es más auditable. Después se pueden evaluar modelos de detección, pero se comparan contra este baseline y se miden retraso y falsos positivos.

## Resumen y comprobación

Una buena alerta incluye métrica, referencia comparable, ventana, umbral, mínimos de calidad, propietario y acción. Monitorizar es diseñar una decisión, no añadir color rojo a un panel.

1. ¿Qué comprobarías antes de atribuir la caída al formulario?
2. ¿Por qué el mismo umbral no sirve necesariamente en lunes y domingo?
3. Escribe un criterio de cierre verificable para el incidente.

## Escala: Parquet, particiones y DuckDB

### Resultado y prerequisites

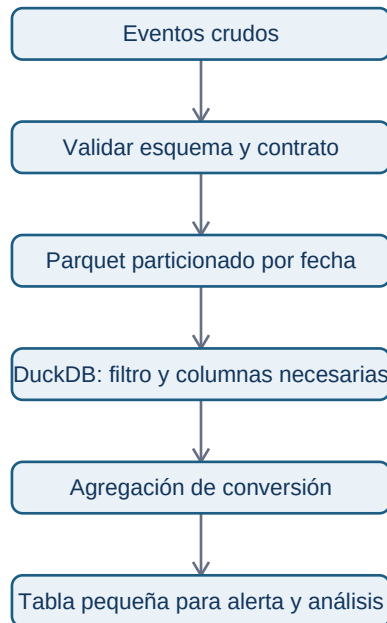
Sabrás decidir por qué una consulta es costosa antes de cambiar de herramienta, y podrás consultar un conjunto Parquet particionado sin leer columnas ni archivos innecesarios. Debes conocer tabla, columna, filtro y agregación.

### El problema antes de la herramienta

Lumen guarda cientos de millones de eventos de visita. Abrir todo en memoria para calcular la conversión de Android en junio es innecesario: la pregunta necesita fecha, plataforma, tipo de evento y usuario; no necesita URL completa, propiedades JSON ni meses distintos. Reducir columnas, filas y transferencias suele ser el primer escalado real.

Un archivo **Parquet** guarda columnas juntas, a diferencia de un archivo de texto que suele recorrer cada fila completa. Esto permite que un motor lea solo las columnas requeridas. Una **partición** divide el conjunto en carpetas o archivos por una clave, por ejemplo `fecha=2026-06-08/plataforma=android/`. No es una sustitución de índices ni una garantía de velocidad: demasiadas particiones pequeñas crean coste de archivos y metadatos.





El diagrama muestra un punto esencial: el formato no decide qué significa una visita ni resuelve duplicados; el contrato se valida antes.

## Consultar sin cargarlo todo

DuckDB es un motor SQL embebido: se ejecuta dentro de un proceso local y puede consultar archivos. Un ejemplo con datos particionados de Lumen es:

```
SELECT
  event_date,
  platform,
  count(DISTINCT user_id) FILTER (WHERE event_name = 'visit') AS visitas,
  count(DISTINCT user_id) FILTER (WHERE event_name = 'booking_confirmed') AS reservas
FROM read_parquet('eventos/event_date=*/platform=*/*.parquet', hive_partitioning = true)
WHERE event_date BETWEEN DATE '2026-06-08' AND DATE '2026-06-14'
  AND platform = 'android'
  AND event_name IN ('visit', 'booking_confirmed')
GROUP BY 1, 2;
```

**Projection pushdown** significa que el motor solicita solo las columnas usadas; **filter pushdown**, que intenta aplicar filtros al leer para saltarse partes irrelevantes. DuckDB documenta ambos comportamientos para Parquet, pero debes confirmar el plan y medir: un filtro sobre una columna no ordenada o archivos sin estadísticas puede no evitar tanta lectura como esperas. `EXPLAIN ANALYZE` es evidencia, no decoración.

## Particionar para preguntas, no por costumbre

Particionar por fecha suele ser útil cuando casi todas las consultas tienen ventana temporal. Añadir `platform` puede ayudar si es un filtro habitual y cada partición sigue teniendo tamaño razonable. Particionar por `user_id` generaría muchísimas carpetas pequeñas: mal patrón para este caso. Revisa tamaño de archivos, coste de listar objetos, evolución de esquema, zona horaria que define `event_date` y retención.

La siguiente estructura hace visible el contrato de lectura:

```
eventos/  
  event_date=2026-06-08/platform=android/part-000.parquet  
  event_date=2026-06-08/platform=ios/part-000.parquet
```

No copies indiscriminadamente datos personales a archivos locales para “ir más rápido”. Conserva permisos, minimiza columnas y usa entornos autorizados. Escala también implica coste, acceso y reproducibilidad.

## Fuentes técnicas actuales

- [DuckDB: lectura de Parquet y pushdown](#)
- [DuckDB: escritura y coste de particiones](#)
- [Apache Parquet](#)

## Resumen y comprobación

Empieza por el grano y la pregunta; después reduce columnas, filas y transferencia. Parquet y DuckDB ayudan cuando su diseño coincide con el acceso, no porque sean etiquetas modernas.

1. ¿Qué columnas son imprescindibles para la consulta de Lumen?
2. ¿Por qué una partición por usuario es mala aquí?
3. ¿Qué evidencia pedirías antes de afirmar que hay pushdown efectivo?

# APIs, datos geoespaciales y fuentes externas

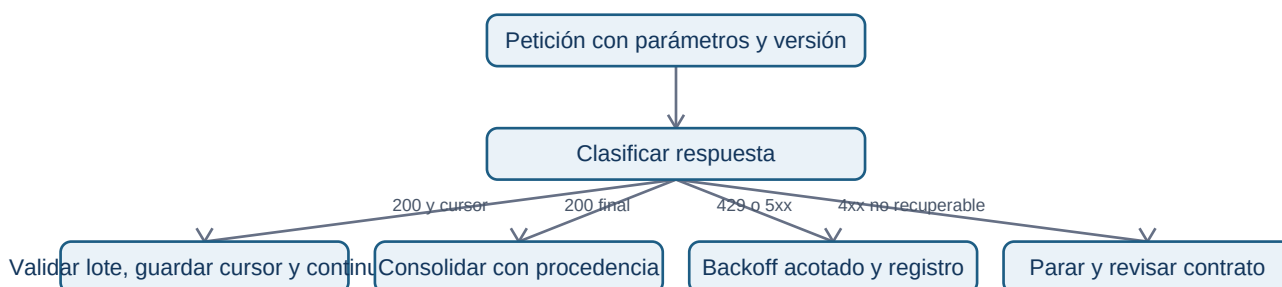
## Resultado y prerequisites

Podrás diseñar una extracción externa pequeña que sea repetible y respetuosa con el proveedor, y evitar dos errores geográficos frecuentes: tratar coordenadas como direcciones y medir distancias con un sistema de referencia inadecuado.

## Pedir datos a otro sistema de forma responsable

Una **API** es una interfaz mediante la que un programa solicita datos o una acción a otro servicio. Para Lumen se quiere unir meteorología pública a reservas por día y zona para investigar una caída. Antes de escribir código, crea un contrato: proveedor y licencia, URL y versión, campos, zona horaria, cobertura, fecha de extracción, propósito, responsable, clave autorizada y política de retención. “Es público” no autoriza cualquier reutilización.

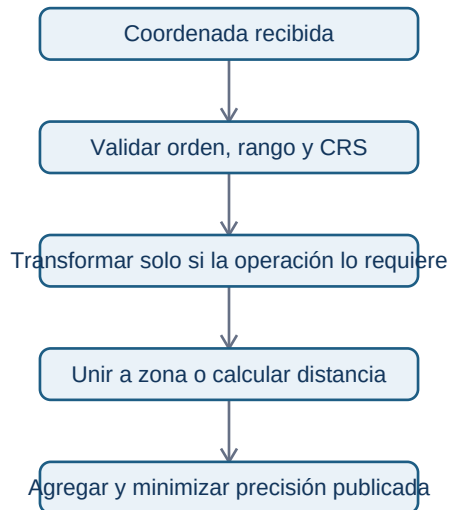
La respuesta puede venir en páginas: el servicio entrega, por ejemplo, 1.000 registros y un cursor para solicitar el siguiente lote. La extracción debe guardar cursor, fecha y respuesta cruda o hash para reproducibilidad. Ante 429 Too Many Requests, espera el tiempo indicado o aplica espera exponencial con límite; no reintentes en bucle ni paralelices hasta derribar el límite. Ante errores 5xx, reintenta un número acotado; ante 4xx de validación, corrige la petición.



El diagrama no es una receta para ignorar términos de uso: cada reintento debe tener límite, registro y dueño. Además, nunca guardes secretos de API en el repositorio; usa un almacén de secretos o variables de entorno autorizadas.

## Coordenadas: número, lugar y sistema de referencia no son sinónimos

Una coordenada 40.4168, -3.7038 es una posición bajo un **sistema de referencia de coordenadas** (CRS). Antes de unirla a barrios o calcular distancia, declara su CRS. EPSG:4326 suele expresar longitud/latitud en grados; los grados no son metros. Calcular distancia euclídea directamente sobre longitud/latitud da una cifra difícil de interpretar y que varía según ubicación. Para medidas métricas locales, transforma a un CRS proyectado apropiado y documenta la decisión.



No infieras hogar, salud, renta o comportamiento a partir de una posición de entrega. Para un panel de Lumen, publicar reservas por celda muy pequeña puede reidentificar a una persona incluso sin nombre. Agrega a zonas con suficiente población, aplica mínimos de conteo, limita acceso y conserva solo la precisión necesaria para la decisión.

## Ejemplo conectado: meteorología no es explicación automática

Tras extraer precipitación diaria por ciudad, Lumen ve menos reservas en días lluviosos. Esa asociación puede servir como variable de contexto para una alerta o una previsión, pero no prueba que la lluvia causó la caída del formulario: puede coincidir con festivo, campaña o cobertura distinta de la fuente. Une por fecha, ciudad, zona horaria y versión de fuente; deja explícita la granularidad perdida al agregar.

## Mini-laboratorio y fuentes técnicas actuales

En el ejercicio integrado diseña una petición paginada y el contrato de una unión geográfica. No necesitas llamar una API real para aprender a diseñarla: evita cargar secretos o datos personales de prueba.

- MDN: [códigos HTTP y 429](#)
- PostGIS: [transformación entre sistemas de referencia](#)
- Open Geospatial Consortium: [CRS](#)

## Resumen y comprobación

Una fuente externa requiere procedencia, límites de uso y validación; una coordenada exige CRS y una decisión de privacidad. Ninguno de los dos convierte una correlación en explicación causal.

1. ¿Qué guardarías para repetir una extracción de API mañana?
2. ¿Por qué no debes calcular kilómetros directamente con longitud y latitud?
3. ¿Qué regla de publicación reduce riesgo de reidentificación en un mapa?